

CS-334: Machine Learning

Lecture 02: Python Crash Course

(with In-Class Exercise #2)

09/01/2026



Follow-Along Notebook at:

<https://github.com/shengpu-tang/CS334-demo>

Course Reminders



- In-Class Exercise #1 – reopened, please submit if you haven't
- HW1 posted, due Sun 9/13
- Notes on calculus and linear algebra on Canvas – Files – Readings

Exercise-01



HW1-Written



HW1-Code



Outline

- Introduction
- Variables and Types
- Operators
- Containers
- Functions
- Control Flow
- Packages
 - Numpy
 - Pandas
 - Matplotlib

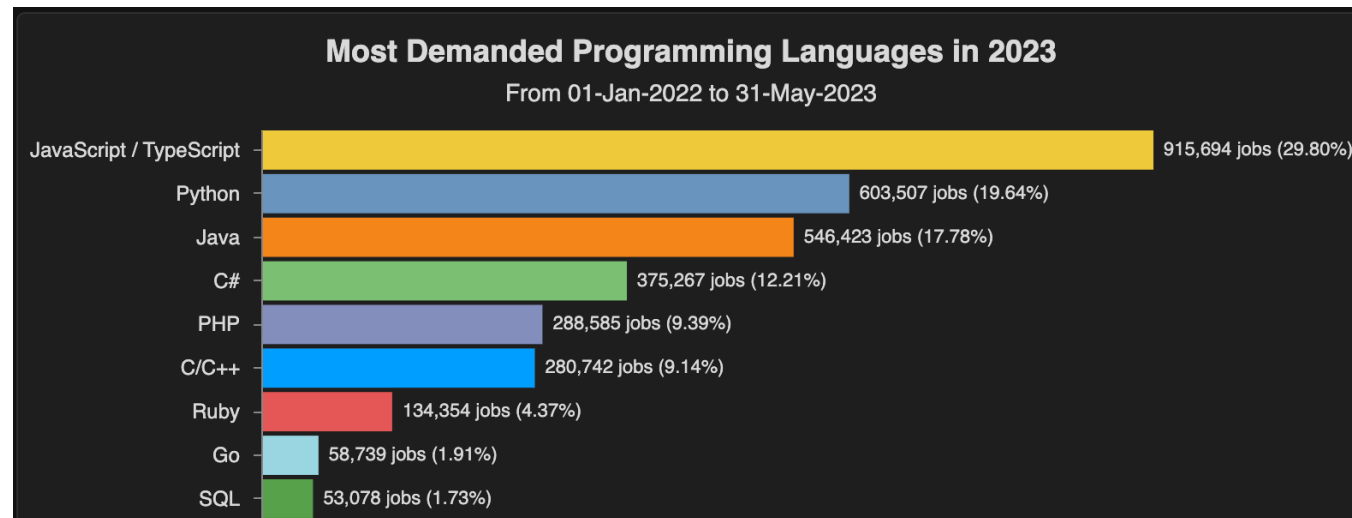
Outline

- **Introduction**
- Variables and Types
- Operators
- Containers
- Functions
- Control Flow
- Packages
 - Numpy
 - Pandas
 - Matplotlib

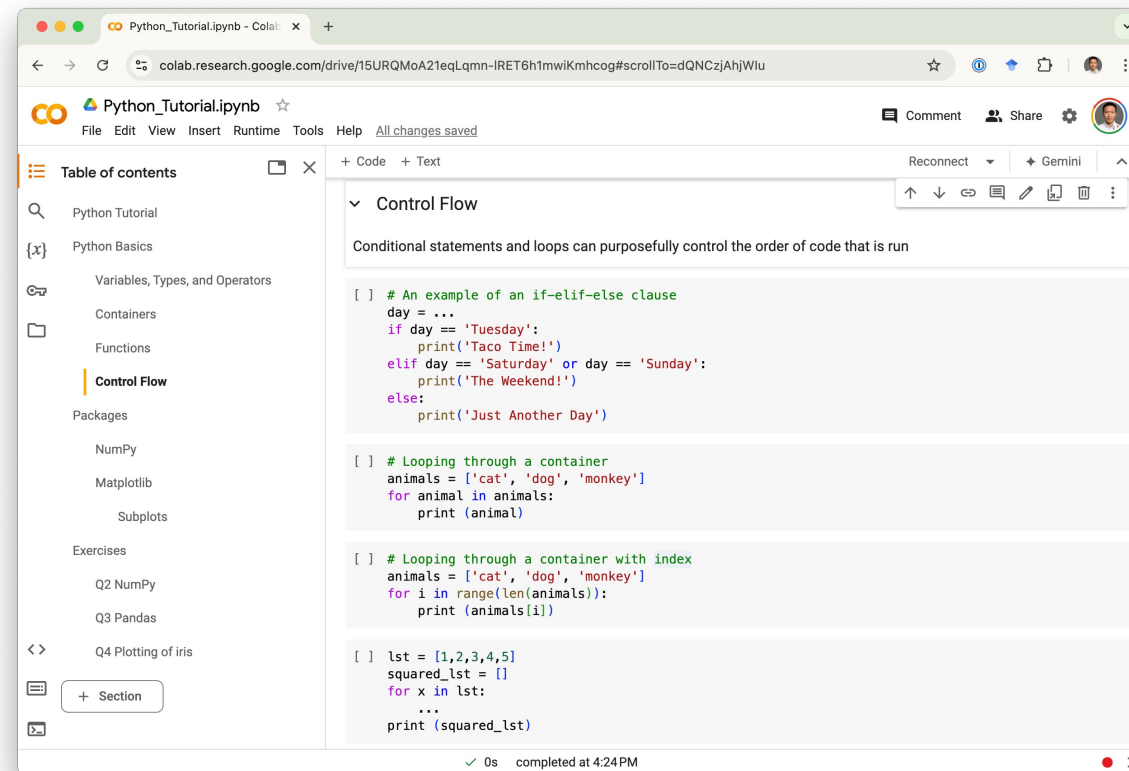
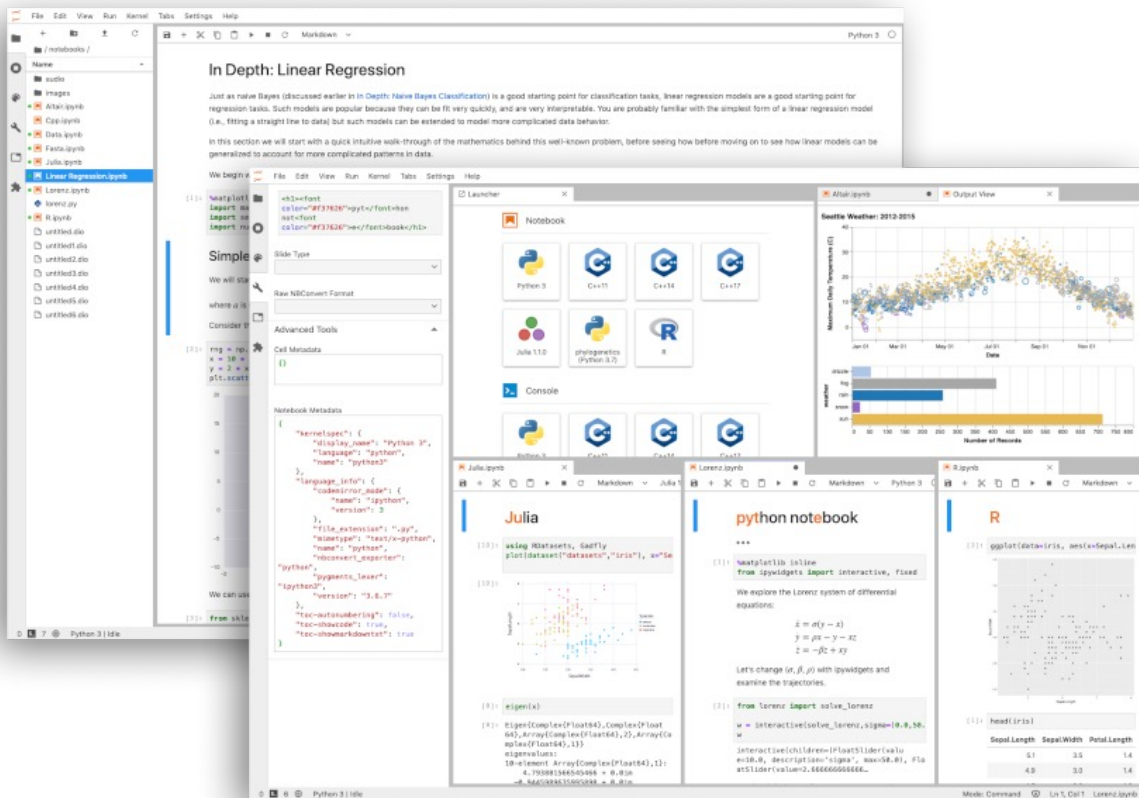


<https://www.devjobsscanner.com/blog/top-8-most-demanded-programming-languages/>

- + Easy to learn
- + Elegant & concise syntax
- + Versatile: lots of scientific computing resources
- Interpreted, not compiled - can be slow



Jupyter / Google Colab



Today

In-Class Exercise #2 on Gradescope (submission closes at 11:59pm)

Slides available on Canvas

<https://www.gradescope.com/courses/1341911>, Entry Code: **X8BBB7**

https://github.com/shengpu-tang/CS334-demo/blob/main/Python_Tutorial.ipynb

Click  Open in Colab

Outline

- Introduction
- **Variables and Types**
- Operators
- Containers
- Functions
- Control Flow
- Packages
 - Numpy
 - Pandas
 - Matplotlib

Variables

- Put data in memory and give it a name.
- Created using the **assignment** operator.

```
x = 3  
print(x)  
3
```

```
x = 3 + 3  
print(x)  
6
```

Variables

```
x = 3
```

Int: Integers

```
π = 3.14
```

Float: Real numbers

```
day = "Tuesday"
```

String: a list of characters.

```
is_tuesday = True
```

Boolean: True/False.

```
# Objects.
```

Comment: Notes for programmers, doesn't get executed.

Outline

- Introduction
- Variables and Types
- **Operators**
- Containers
- Functions
- Control Flow
- Packages
 - Numpy
 - Pandas
 - Matplotlib

Operators

Operator	Python	Python Example
Addition	+	1 + 2
Subtraction	-	2 - 1
Multiplication	*	2 * 4
Division	/	4 / 2
Modulo	%	4 % 3
Equal	==	2 == 3
Less Than / Less Than Equal	<, <=	2 < 3
Greater Than / Greater than Equal	>, >=	2 > 3
And	and	x < 5 and x > 2
Or	or	x < 5 or x > 2
Not	not	not (x < 5)

Two kinds of divide

```
print(17 / 3)  
5.666666666666667
```

```
print(17 // 3)  
5
```

```
print(17 % 3)  
2
```

Exercise 2.1 Float Division

```
print(11.0 // 2.5)
```

You can add strings too!

```
hello = 'hello'
world = "world"
hw = hello + ' ' + world    # String concatenation
print(hw)                   # prints "hello world"
```

Exercise 2.2 “Add” string and number

Outline

- Introduction
- Variables and Types
- Operators
- **Containers**
- Functions
- Control Flow
- Packages
 - Numpy
 - Pandas
 - Matplotlib

List

```
index:  0    1    2    3    4    5  
x = [1, 2, 3, 'a', 'b', 'c']
```

```
x[0]    # zero-indexed
```

```
x.append(100) # add item to the end
```

```
x[3] = 'A'   # modify item
```

```
x[2:5] # slice operator
```

Common mistake: copying lists

```
first_list = [1, 2, 3]
second_list = first_list    # try to copy
second_list[2] = 100        # change an element
print(first_list)
[1, 2, 100]
print(second_list)
[1, 2, 100]
```

Common mistake: copying lists

```
first_list = [1, 2, 3]
second_list = first_list[:] # make a copy
second_list[2] = 100        # change an element
print(first_list)
[1, 2, 3]
print(second_list)
[1, 2, 100]
```

Dictionary

Just like lists but elements can be indexed by non-integers.

```
a = {}      # Empty dict.  
a["key"] = "value"  
print(a)  
{"key": "value"}
```

Exercise 2.3 Copying Dictionary

Outline

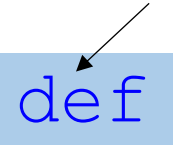
- Introduction
- Variables and Types
- Operators
- Containers
- **Functions**
- Control Flow
- Packages
 - Numpy
 - Pandas
 - Matplotlib

Functions

```
def hours_and_minutes(minutes):  
    n_hours = minutes // 60  
    n_minutes = minutes % 60  
    return n_hours, n_minutes
```

Functions

About to define a function



```
def hours_and_minutes(minutes):  
    n_hours = minutes // 60  
    n_minutes = minutes % 60  
    return n_hours, n_minutes
```

Functions

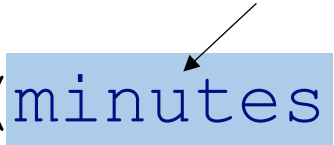
Name of the function

```
def hours_and_minutes(minutes):  
    n_hours = minutes // 60  
    n_minutes = minutes % 60  
    return n_hours, n_minutes
```

Functions

Input arguments

```
def hours_and_minutes(minutes):  
    n_hours = minutes // 60  
    n_minutes = minutes % 60  
    return n_hours, n_minutes
```



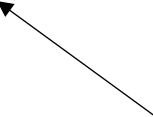
Functions

```
def hours_and_minutes(minutes):  
    n_hours = minutes // 60  
    n_minutes = minutes % 60  
    return n_hours, n_minutes
```

Body of the function
Whitespaces/tabs matter!
They define the scope.

Functions

```
def hours_and_minutes(minutes):  
    n_hours = minutes // 60  
    n_minutes = minutes % 60  
    return n_hours, n_minutes
```



Return value, what we get out

Outline

- Introduction
- Variables and Types
- Operators
- Containers
- Functions
- **Control Flow**
- Packages
 - Numpy
 - Pandas
 - Matplotlib

If

```
if age > 21:  
    print("Come in!")  
elif age == 20:  
    print("Try again next year.")  
else:  
    print("You've got some time!")
```

Condition Statement. Must result in True/False

If/Elif Block. Evaluated if condition is true.

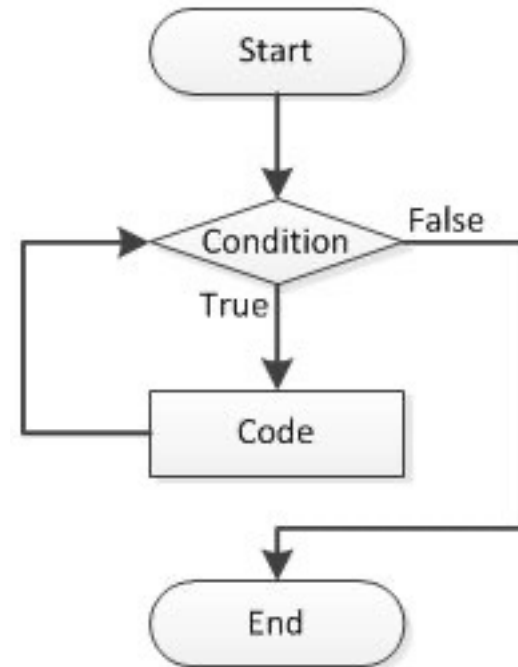
Else Block.
Evaluated if no conditions are true.

COLONS

While-Loop

```
i = 0
while i < 3:
    print(i)
    i += 1
print(i)
```

0
1
2
3



For-Loop

```
animals = ['cat', 'dog', 'monkey']  
for animal in animals:  
    print(animal)
```

cat

dog

monkey

Loop through items in a container directly

For-Loop

```
animals = ['cat', 'dog', 'monkey']  
for i in range(len(animals)):  
    print(animals[i])
```

cat

dog

monkey

Loop through indices

Advanced Usage: List Comprehension

```
x = [1, 2, 3]
y = []
for item in x:
    y.append(item*item)
print(y)
```

[1, 4, 9]

```
x = [1, 2, 3]
y = [item*item for item in x]
print(y)
```

[1, 4, 9]

Outline

- Introduction
- Variables and Types
- Operators
- Containers
- Functions
- Control Flow
- **Packages**
 - **Numpy**
 - **Pandas**
 - **Matplotlib**

Modules

A module is a collection of variables, functions, classes.
Also called: library, package

```
import math  
print(math.sqrt(16))
```

```
from math import sqrt  
print(sqrt(16))
```

```
import math as m  
print(m.sqrt(16))
```

NumPy

- Tensor/matrix operation library.
- Lists, but more dimensions, and faster.
- NOTE: numpy is pre-installed on colab. Normally you would need to install it manually.

```
import numpy as np
x = np.array([1, 2, 3])          # vector
A = np.array([[1, 0, 0], [0, 1, 1]]) # 2x3 matrix
A_ = A.T                        # transposed 3x2 matrix
```

Element-wise operations

```
x = np.array([1, 2, 3])
```

```
print(x + 1)  
[2 3 4]
```

```
print(x * 2)  
[2 4 6]
```

```
print(x ** 2)  
[1 4 9]
```

```
print(x > 1)  
[False  True  True]
```

```
print(x == 1)  
[ True False False]
```

Dot Product; Matrix-Vector Product

```
x = np.array([1, 2, 3])
```

```
A = np.array([[1, 0, 0], [0, 1, 1]])
```

```
print(x @ x)
```

```
print(np.dot(x, x))
```

```
print(x.dot(x))
```

14

14

14

```
print(A @ x)
```

```
print(np.dot(A, x))
```

```
print(A.dot(x))
```

[1 5]

[1 5]

[1 5]

NumPy Summary Statistics

```
x = np.array([1, 2, 3])
```

```
print(np.sum(x))
```

```
6
```

```
print(np.mean(x))
```

```
2.0
```

```
print(np.std(x))
```

```
0.816496580927726
```

```
A = np.array([[1, 0, 0], [0, 1, 1]])
```

```
print(np.sum(A, axis=0))
```

```
[1 1 1]
```

```
print(np.mean(A, axis=0))
```

```
[0.5 0.5 0.5]
```

```
print(np.std(A, axis=0))
```

```
[0.5 0.5 0.5]
```

NumPy

Exercise 3.1 shape vs size

Exercise 3.2 Multiplication

Exercise 3.3 numpy dtype

```
arr = np.array([1,2,3])  
arr[0] = 1.5 # trying to change 1 to 1.5  
print(arr)
```

```
[1 2 3]
```

Pandas DataFrame

```
import pandas as pd
df = pd.DataFrame([
    [1, 1, 'odd'], [2, 4, 'even'],
    [3, 9, 'odd'], [4, 16, 'even']],
    columns=['x', 'x^2', 'parity'])
df
```

	x	x^2	parity
0	1	1	odd
1	2	4	even
2	3	9	odd
3	4	16	even

- A table just like a 2D NumPy array -- but
- Rows/columns have names
- Table can hold non-numbers

Pandas DataFrame

```
df[df['parity'] == 'odd']
```

Slice operator based on logical condition

	x	x^2	parity
0	1	1	odd
2	3	9	odd

```
df[df['parity'] == 'odd']['x'].mean()
```

2.0

Chaining slice operator with summary statistics

Matplotlib

- Package for plotting data.

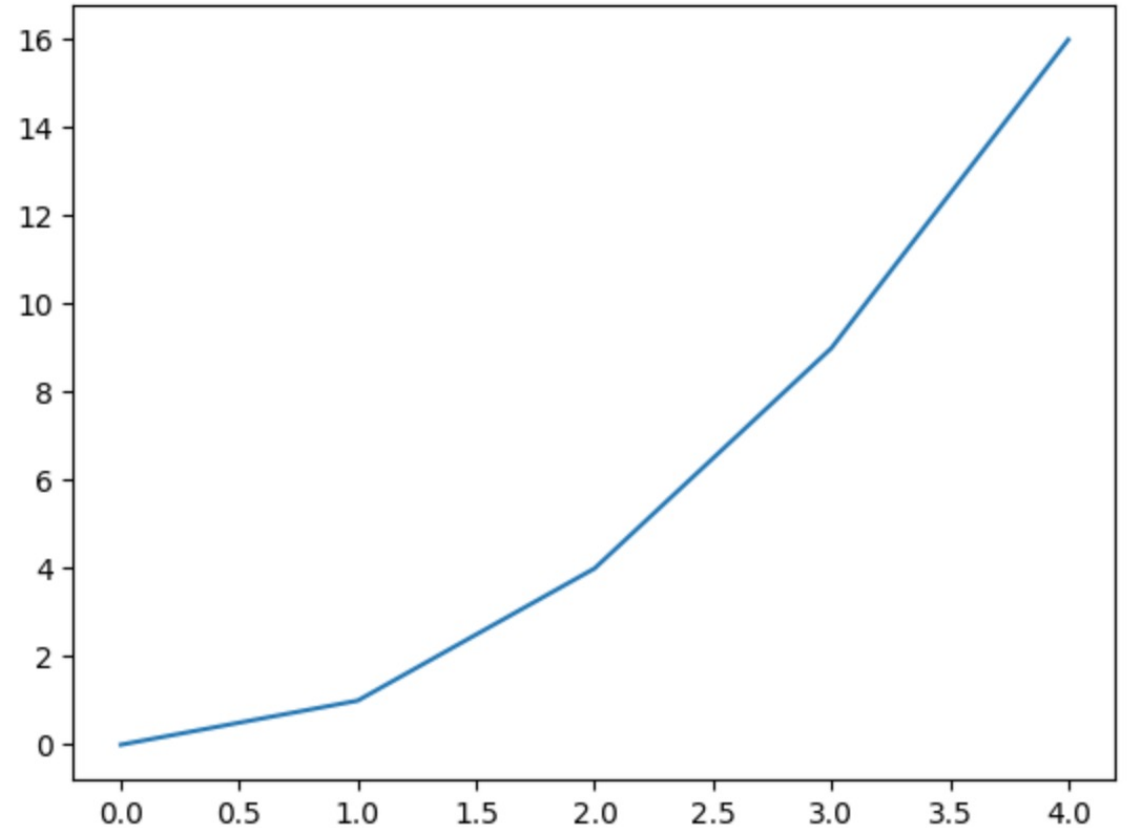
```
import matplotlib.pyplot as plt
```

```
x = np.arange(5)
```

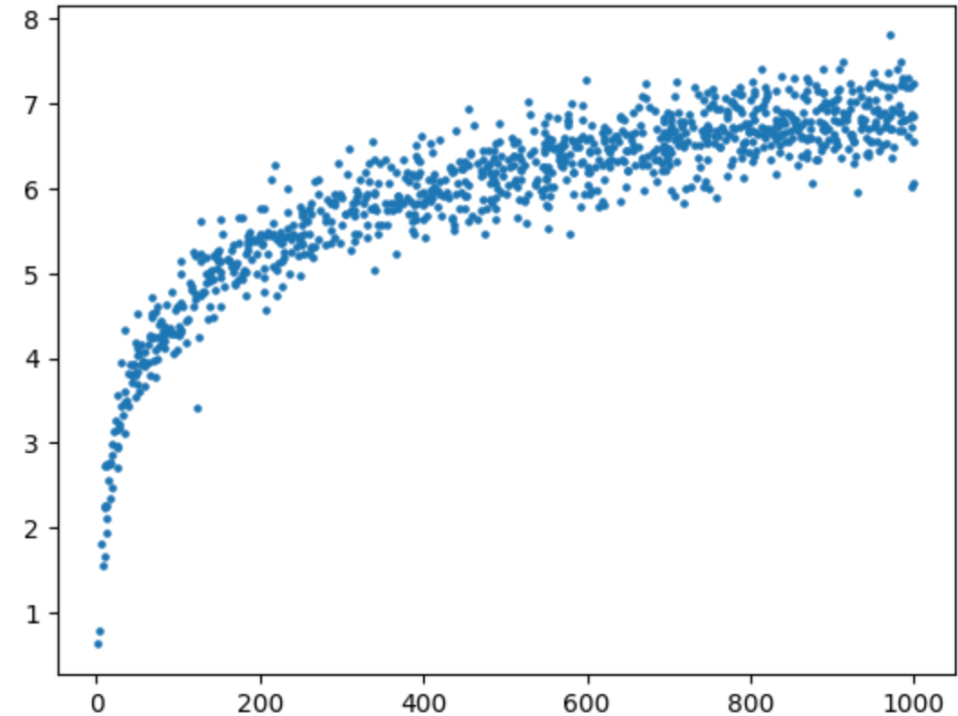
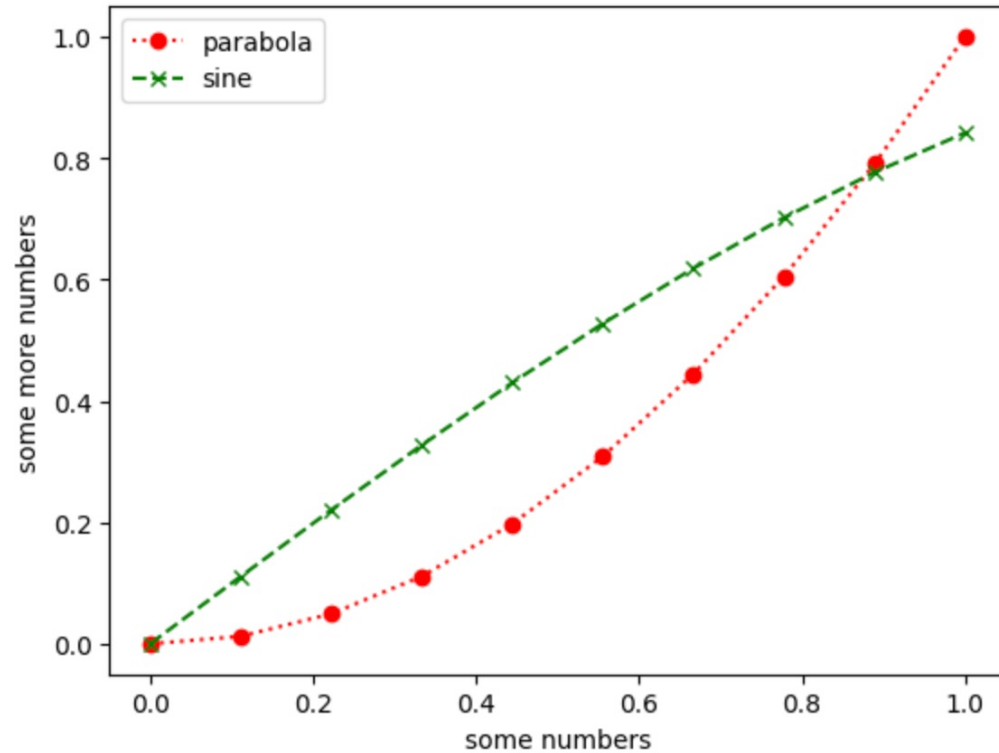
```
y = x * x
```

```
plt.plot(x, y)
```

```
plt.show()
```



Matplotlib styling



Pandas + Matplotlib

Exercise 4 Data Exploration

Search for files 0 items selected

- ▼ CS-334-2: Machine Learning
 - ▶ Course Logistics
 - ▶ Homeworks
 - ▼ In-Class Exercises
 - ▶ Lectures

Name ▲	Date Created	Date Modified
 iris_cs334.csv	Jan 6, 2025	Aug 28, 2024

CO Python_Tutorial.ipynb ☆
File Edit View Insert Runtime Tool

Files **Upload button**

{x} ..

▶ sample_data



File picker

Pandas + Matplotlib

Exercise 4 Data Exploration

	sepal_length	sepal_width	petal_length	petal_width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
95	4.9	3.1	1.5	0.2	Setosa
96	5.8	2.7	5.1	1.9	Virginica
97	6.8	3.2	5.9	2.3	Virginica
98	6.2	3.4	5.4	2.3	Virginica
99	5.9	3.0	5.1	1.8	Virginica

100 rows x 5 columns

